Restaurant SaaS – Vollständige Funktionsübersicht

> ***Stand:*** *05.04.2026 |* ***Phasen 1–3 abgeschlossen*** *| Phase 4 (SaaS-Features) steht als Nächstes an*

Inhaltsverzeichnis

1. Projektübersicht
2. Tech-Stack
3. Projekt starten
4. Wo wird was gespeichert?
5. Test-Zugangsdaten
6. Alle Funktionen im Detail
   - Login & Authentifizierung
   - Dashboard
   - Bestellungen
   - Tischplan & QR-Codes
   - Speisekarte
   - Reservierungen
   - Mitarbeiter
   - Gäste-Bestellseite
   - Live-Bestellstatus
7. Datenbank-Schema
8. API-Endpunkte
9. Sicherheit & DSGVO
10. Projektstruktur

Projektübersicht

Eine Multi-Tenant Restaurant-Management-Software (SaaS). Jedes Restaurant ist ein eigener Tenant – alle Daten werden über eine `restaurant_id` voneinander getrennt.

Was kann die App aktuell?

- Mitarbeiter-Login mit Rollen (Admin, Kellner, Küche)
- Dashboard mit Live-Statistiken und Charts
- Bestellungen verwalten mit Live-Updates (Socket.io)
- Tischplan mit Status-Management und QR-Code-Generierung
- Speisekarte verwalten (Kategorien + Gerichte)
- Reservierungen mit Kalenderansicht
- Mitarbeiter anlegen, bearbeiten, deaktivieren
- Gäste bestellen per QR-Code am Tisch
- Live-Bestellstatus für Gäste nach der Bestellung

Tech-Stack

Bereich	Technologie
Backend	Node.js + Express + TypeScript

Frontend	React 18 + TypeScript + Tailwind CSS + Vite
Datenbank	PostgreSQL (gehostet auf Supabase)
Echtzeit	Socket.io (WebSocket)
Auth	JWT (JSON Web Tokens) + bcrypt
State	Zustand (Frontend-Store)
Charts	Recharts
QR-Codes	qrcode.react

Projekt starten

Voraussetzungen

- Node.js (Version 20+)
- PostgreSQL (oder Supabase-Account)

Backend starten

```
cd restaurant-app/backend
npm install
npm run dev
# → Läuft auf http://localhost:3001
```

Frontend starten

```
cd restaurant-app/frontend
npm install
npm run dev
# → Läuft auf http://localhost:5173
```

Datenbank einrichten

```
# Schema erstellen (alle Tabellen)
psql $DATABASE_URL -f restaurant-app/database/schema.sql

# Testdaten laden (Demo-Restaurant + Testnutzer)
psql $DATABASE_URL -f restaurant-app/database/seed.sql
```

Wo wird was gespeichert?

Login & Session

Was	Wo	Details

JWT-Token (nach Login)	Browser `localStorage`	Schlüssel: `restaurant-auth` – enthält Token + Mitarbeiter-Daten
Passwörter	PostgreSQL `mitarbeiter.passwort_hash`	bcrypt-Hash (12 Runden) – Klartext wird **nie** gespeichert
JWT-Secret	Backend `.env` Datei	Variable `JWT_SECRET` – wird zum Signieren/Verifizieren verwendet
Token-Gültigkeit	Backend `.env`	Variable `JWT_EXPIRES_IN` – Standard: `7d` (7 Tage)

Datenbank-Verbindung

Was	Wo	Details
DB-Verbindung	`restaurant-app/backend/.env`	Variable `DATABASE_URL` (PostgreSQL Connection String)
Supabase-Keys	`restaurant-app/frontend/src/lib/supabase.ts`	`SUPABASE_URL` + `SUPABASE_ANON_KEY` (hardcoded, öffentlicher Key)

Wichtige Dateien im Backend

Datei	Zweck
`backend/.env`	Alle Umgebungsvariablen (DB-URL, JWT-Secret, Port, Frontend-URL)
`backend/src/models/db.ts`	Datenbank-Verbindung (PostgreSQL Pool)
`backend/src/middleware/auth.ts`	JWT-Verifizierung + Rollen-Check
`backend/src/server.ts`	Express-Server + Socket.io Setup

Wichtige Dateien im Frontend

Datei	Zweck
`frontend/src/store/auth.ts`	Zustand-Store für Login-Status (persistiert in localStorage)
`frontend/src/lib/api.ts`	API-Client (fügt JWT-Token automatisch an jeden Request an)
`frontend/src/lib/supabase.ts`	Supabase-Client für Datenbankzugriff
`frontend/src/lib/utils.ts`	Hilfsfunktionen (Preisformatierung, Status-Labels, Farben)

Wo liegen die Daten?

Daten	Tabelle	Wichtige Felder
Restaurants	`restaurants`	Name, Logo, Währung, Abo-Status, Lizenzcode

Mitarbeiter/Logins	`mitarbeiter`	Email, Passwort-Hash, Rolle, aktiv/inaktiv
Tische	`tische`	Nummer, Kapazität, Status, QR-URL
Speisekarte	`gerichte` + `kategorien`	Name, Preis, Beschreibung, Allergene, verfügbar
Bestellungen	`bestellungen` + `bestellpositionen`	Tisch, Status, Gesamtpreis, einzelne Gerichte
Reservierungen	`reservierungen`	Gastname, Telefon, Datum, Personen, Status

---

Test-Zugangsdaten

Nach dem Ausführen von `seed.sql` sind diese Accounts verfügbar:

Email	Passwort	Rolle	Berechtigungen
`admin@demo.de`	`test1234`	Admin	Alles (Tische, Speisekarte, Mitarbeiter, Bestellungen, Reservierungen)
`kellner@demo.de`	`test1234`	Kellner	Bestellungen, Tischplan, Reservierungen
`kueche@demo.de`	`test1234`	Küche	Bestellungen (Status ändern)

Demo-Modus: Auf der Login-Seite gibt es einen "Demo ansehen"-Button – funktioniert komplett ohne Datenbank mit Beispieldaten.

Demo-Restaurant: "Trattoria Demo" mit 5 Tischen, 8 Gerichten in 4 Kategorien.

Alle Funktionen im Detail

1. Login & Authentifizierung

Seite: `/login` **Datei:** `frontend/src/pages/Login.tsx`

Was passiert beim Login:

1. Nutzer gibt Email + Passwort ein
2. Supabase authentifiziert den Nutzer
3. Mitarbeiter-Daten (Name, Rolle, Restaurant) werden aus der DB geladen
4. JWT-Token + Mitarbeiterdaten werden im Browser- `localStorage` gespeichert (Schlüssel: `restaurant-auth`)
5. Weiterleitung zum Dashboard

Sicherheit:

- Passwörter werden mit bcrypt (12 Runden) gehasht
- JWT-Token enthält: `mitarbeiterId` , `restaurantId` , `rolle`
- Token läuft nach 7 Tagen ab
- Bei ungültigem Token → automatische Weiterleitung zur Login-Seite
- Passwort kann ein-/ausgeblendet werden (Auge-Icon)

Rollen-System:

Rolle	Zugriff
`admin`	Alles – kann Mitarbeiter, Speisekarte, Tische verwalten
`kellner`	Bestellungen, Tischplan, Reservierungen
`kueche`	Nur Bestellungen (Status ändern)

---

2. Dashboard

Seite: `/dashboard` **Datei:** `frontend/src/pages/Dashboard.tsx`

5 Statistik-Karten (oben):

Karte	Zeigt	Farbe
Offene Bestellungen	Anzahl mit Status "offen"	Amber
In Zubereitung	Anzahl mit Status "in_zubereitung"	Orange
Tagesumsatz	Summe aller servierten/bezahlten Bestellungen	Grün
Reservierungen	Aktive Reservierungen heute	Blau
Tische belegt	z.B. "3/5" (besetzt/gesamt)	Violett

Charts (Mitte):

- **Umsatz-Chart** (2/3 Breite) – Balkendiagramm mit Tagesumsatz
- **Auslastung-Donut** (1/3 Breite) – Kreisdiagramm der Tisch-Belegung

Listen (unten, 3 Spalten):

- **Bestellungs-Verteilung** – Kreisdiagramm nach Status
- **Aktive Bestellungen** – Die letzten 5 offenen Bestellungen mit Tisch-Nr, Gerichten, Preis
- **Reservierungen heute** – Nächste 5 Reservierungen mit Uhrzeit, Name, Personenzahl

---

3. Bestellungen

Seite: `/bestellungen` **Datei:** `frontend/src/pages/Bestellungen.tsx`

Funktionen:

- Alle aktiven Bestellungen als Karten-Grid
- Jede Karte zeigt: Tischnummer, bestellte Gerichte mit Mengen, Gesamtpreis, Anmerkung
- **Status-Workflow:** Offen → In Zubereitung → Serviert → Bezahlt
- Ein Klick auf den Status-Button schiebt die Bestellung zum nächsten Status
- **Live-Updates** über Socket.io – neue Bestellungen erscheinen sofort

Bestellungs-Status-Farben:

Status	Farbe
Offen	Gelb
In Zubereitung	Orange

Serviert	Grün
Bezahlt	Grau (verschwindet aus der aktiven Liste)

---

4. Tischplan & QR-Codes

Seite: `/tischplan` **Datei:** `frontend/src/pages/Tischplan.tsx`

Tisch-Verwaltung:

- Grid-Ansicht aller Tische (2–5 Spalten, responsive)
- Jeder Tisch zeigt: Nummer (z.B. "T1"), Kapazität, aktueller Status
- **Status-Wechsel per Klick:** Frei → Besetzt → Wartet auf Zahlung → Frei
- Status-Zusammenfassung oben (Badges: Frei, Besetzt, Zahlung, Gesamt)

Tisch-CRUD (nur Admin):

- Neuen Tisch anlegen (Nummer + Kapazität)
- Tisch bearbeiten (Nummer + Kapazität ändern)
- Tisch löschen (mit Bestätigungsdialog)

QR-Code-Feature:

- Jeder Tisch hat einen QR-Code-Button
- Klick öffnet ein Modal mit:
  - **Echtem QR-Code-Bild** (scanbar)
  - Bestell-Link als Text
  - **"Link kopieren"** Button
  - **"Drucken"** Button – öffnet Druckfenster für einzelnen QR-Code
- **"QR-Codes drucken"** Button in der Topbar:
  - Zeigt Vorschau aller QR-Codes
  - **"Alle drucken"** – druckt alle QR-Codes auf einmal (3er-Raster, druckoptimiert)
  - Jeder QR-Code zeigt Tischnummer + "Zum Bestellen QR-Code scannen"

QR-Code-URL-Format:

```
https://deine-domain.de/bestellen/{restaurantId}/{tischId}
```

---

5. Speisekarte

Seite: `/speisekarte` **Datei:** `frontend/src/pages/Speisekarte.tsx`

Kategorie-Verwaltung:

- Kategorien anlegen (z.B. Vorspeisen, Hauptgerichte, Desserts, Getränke)
- Kategorien umbenennen
- Leere Kategorien löschen (Kategorien mit Gerichten können nicht gelöscht werden)
- Reihenfolge wird berücksichtigt

Gerichte-Verwaltung:

- Neues Gericht anlegen: Name, Beschreibung, Preis, Kategorie, Allergene
- Gericht bearbeiten
- Gericht löschen

- **Verfügbarkeit ein/aus** – Toggle-Schalter pro Gericht (nicht verfügbare Gerichte werden Gästen nicht angezeigt)

Anzeige:

- Gerichte gruppiert nach Kategorien
- Jede Gericht-Karte zeigt: Name, Beschreibung, Preis (in EUR formatiert), Allergene, Verfügbarkeit

Preise:

- Werden **immer** serverseitig validiert (nie vom Client übernommen)
- Format: Euro mit 2 Dezimalstellen (z.B. 13,90 EUR)

---

6. Reservierungen

Seite: `/reservierungen` **Datei:** `frontend/src/pages/Reservierungen.tsx`

Kalender-Navigation:

- **Wochenleiste** oben – 7 Tage anklickbar
- Vor/Zurück-Buttons für Tagesnavigation
- "Heute"-Button zum schnellen Zurückspringen

Tagesstatistiken:

- Anzahl Reservierungen für den gewählten Tag
- Gesamtanzahl Personen

Reservierungs-Liste:

- Jede Reservierung zeigt: Gastname, Telefonnummer, Uhrzeit, Personenzahl, Status, Anmerkung
- **Status-Buttons:** Ausstehend → Bestätigt / Storniert
- **Neue Reservierung** anlegen: Gastname, Telefon, Datum/Uhrzeit, Personen, Tisch (optional), Anmerkung

Status-Farben:

Status	Farbe
Ausstehend	Gelb
Bestätigt	Grün
Storniert	Grau

---

7. Mitarbeiterverwaltung

Seite: `/mitarbeiter` (nur Admin) **Datei:** `frontend/src/pages/Mitarbeiter.tsx`

Funktionen:

- **Mitarbeiter anlegen:** Name, Email, Passwort, Rolle (Admin/Kellner/Küche)
- **Mitarbeiter bearbeiten:** Name und Rolle ändern
- **Passwort zurücksetzen:** Neues Passwort vergeben
- **Deaktivieren/Aktivieren:** Deaktivierte Mitarbeiter können sich nicht mehr einloggen

Filter-Tabs:

- Aktiv (Standard)
- Inaktiv
- Alle

Anzeige pro Mitarbeiter:

- Name, Email, Rolle (farbiger Badge), aktiv/inaktiv Status
- Erstellt-am-Datum

Rollen-Badge-Farben:

Rolle	Farbe
Admin	Violett
Kellner	Himmelblau
Küche	Amber

---

8. Gäste-Bestellseite

Seite: `/bestellen/:restaurantId/:tischId` (öffentlich, kein Login nötig) **Datei:** `frontend/src/pages/Bestellen.tsx`

So funktioniert es:

1. Gast scannt QR-Code am Tisch mit dem Handy
2. Speisekarte wird geladen (nur verfügbare Gerichte, gruppiert nach Kategorien)
3. Gast wählt Gerichte aus (+ / - Buttons für Mengen)
4. Warenkorb zeigt: gewählte Gerichte, Mengen, Einzelpreise, Gesamtpreis
5. Optionale Anmerkung (z.B. "Ohne Zwiebeln")
6. "Bestellen" Button schickt die Bestellung ab

Nach der Bestellung:

- Tisch wird automatisch auf "besetzt" gesetzt
- Bestellung erscheint sofort bei den Mitarbeitern (Socket.io)
- Gast sieht den Live-Status-Tracker (siehe nächster Punkt)

Wichtig: Preise werden **serverseitig** aus der Datenbank geladen – der Client kann keine Preise manipulieren.

---

9. Live-Bestellstatus für Gäste

Komponente: `frontend/src/components/gaeste/BestellStatusTracker.tsx` **Socket-Hook:** `frontend/src/hooks/useGaesteSocket.ts`

Nachdem ein Gast bestellt hat, sieht er statt der Erfolgsmeldung einen **Live-Status-Tracker**:

4-Schritte-Fortschrittsleiste:

```
[1] Offen → [2] In Zubereitung → [3] Serviert → [4] Bezahlt
```

- Aktiver Schritt ist rot hervorgehoben mit Ring
- Erledigte Schritte zeigen ein Häkchen

- Zukünftige Schritte sind grau

Zu jedem Status gehört eine Nachricht:

Status	Nachricht
Offen	"Deine Bestellung ist eingegangen."
In Zubereitung	"Dein Essen wird gerade zubereitet!"
Serviert	"Dein Essen wurde serviert. Guten Appetit!"
Bezahlt	"Bezahlt – vielen Dank für deinen Besuch!"

So funktioniert die Echtzeit-Aktualisierung:

1. Nach der Bestellung verbindet sich das Handy des Gastes per Socket.io (ohne Login)
2. Der Socket tritt dem Tisch-Raum bei (`tisch:{restaurantId}:{tischId}`)
3. Wenn ein Mitarbeiter den Bestellstatus ändert, wird das Event an den Tisch-Raum gesendet
4. Der Status-Tracker aktualisiert sich **automatisch** – kein Neuladen nötig
5. Unten steht: "Live-Updates aktiv – diese Seite aktualisiert sich automatisch" (pulsierend)

Gesamtpreis wird unter dem Tracker in einer Karte angezeigt.

Datenbank-Schema

8 Tabellen, alle mit UUID-Primary-Keys:

```
restaurants          ← Tenant-Tabelle (ein Eintrag pro Restaurant)
├── kategorien       ← Speisekarte-Kategorien
├── tische           ← Tische mit Status + QR-URL
├── gerichte         ← Gerichte mit Preisen + Allergenen
├── bestellungen     ← Bestellungen mit Status
│   └── bestellpositionen  ← Einzelne Gerichte pro Bestellung
├── reservierungen   ← Gast-Reservierungen
└── mitarbeiter      ← Login-Accounts mit Rollen
```

Multi-Tenant: Jede Tabelle hat eine `restaurant_id` Spalte. Alle Abfragen filtern danach.

Automatische Timestamps: `erstellt_am` wird automatisch gesetzt. Bei Bestellungen gibt es zusätzlich `aktualisiert_am` (wird per Trigger aktualisiert).

API-Endpunkte

Öffentlich (kein Login)

Methode	Pfad	Beschreibung
POST	`/api/auth/login`	Login (Email + Passwort)
GET	`/api/speisekarte?restaurant_id=...`	Speisekarte laden (für Gäste)
POST	`/api/bestellungen`	Bestellung abschicken (Gäste)

POST	`/api/reservierungen`	Reservierung erstellen

Authentifiziert (JWT erforderlich)

Methode	Pfad	Rolle	Beschreibung
GET	`/api/bestellungen`	Alle	Aktive Bestellungen
PATCH	`/api/bestellungen/:id/status`	Alle	Bestellstatus ändern
GET	`/api/tische`	Alle	Alle Tische
POST	`/api/tische`	Admin	Tisch anlegen
PATCH	`/api/tische/:id`	Admin	Tisch bearbeiten
PATCH	`/api/tische/:id/status`	Admin, Kellner	Tisch-Status ändern
DELETE	`/api/tische/:id`	Admin	Tisch löschen
POST	`/api/speisekarte`	Admin	Gericht anlegen
PATCH	`/api/speisekarte/:id`	Admin	Gericht bearbeiten
DELETE	`/api/speisekarte/:id`	Admin	Gericht löschen
GET	`/api/reservierungen`	Alle	Reservierungen laden
PUT	`/api/reservierungen/:id`	Alle	Reservierung bearbeiten
DELETE	`/api/reservierungen/:id`	Admin	Reservierung löschen
GET	`/api/mitarbeiter`	Admin	Alle Mitarbeiter
POST	`/api/mitarbeiter`	Admin	Mitarbeiter anlegen
PATCH	`/api/mitarbeiter/:id`	Admin	Mitarbeiter bearbeiten
PATCH	`/api/mitarbeiter/:id/passwort`	Admin	Passwort ändern

Socket.io Events

Event	Richtung	Beschreibung
`raum_beitreten`	Client → Server	Mitarbeiter tritt Restaurant-Raum bei
`tisch_beitreten`	Client → Server	Gast tritt Tisch-Raum bei
`neue_bestellung`	Server → Mitarbeiter	Neue Bestellung eingegangen
`bestellung_aktualisiert`	Server → Mitarbeiter + Gast	Bestellstatus geändert

Sicherheit & DSGVO

Sicherheit

- **Passwörter:** bcrypt mit 12 Runden (nie im Klartext gespeichert)
- **JWT:** Wird bei jedem API-Request im `Authorization: Bearer` -Header mitgeschickt
- **Multi-Tenant:** `restaurant_id` wird aus dem JWT extrahiert, nie aus dem Request-Body
- **Preisvalidierung:** Gesamtpreise werden serverseitig berechnet, nie vom Client übernommen
- **Automatischer Logout:** Bei abgelaufenem Token wird der Nutzer zur Login-Seite weitergeleitet

DSGVO

- **Personenbezogene Daten:**
  - `reservierungen.telefon` – Löschfrist: 30 Tage nach Reservierungsdatum
  - `mitarbeiter.email` – Personenbezogen, wird geschützt gespeichert
- **Passwort-Hashes** werden nie in API-Responses zurückgegeben
- DSGVO-Prüfprotokoll in `project/dsgvo-log.md`
- Verarbeitungsverzeichnis in `project/dsgvo-datenkatalog.md`

Projektstruktur

```
restaurant-saas/
├── README.md                    ← Diese Datei
├── CLAUDE.md                    ← Automatische Regeln für Claude
├── LIES_MICH_ZUERST.md          ← Schnellstart & Rollenübersicht
│
├── project/                     ← Projektdokumentation
│   ├── status.md                ← Aktueller Projektstand
│   ├── todos.md                 ← Aufgabenliste nach Phasen
│   ├── entscheidungen.md        ← Getroffene Entscheidungen
│   ├── dsgvo-log.md             ← DSGVO-Prüfprotokoll
│   └── dsgvo-datenkatalog.md    ← Verarbeitungsverzeichnis
│
├── datenstruktur/               ← Technische Dokumentation
│   ├── datenbank-schema.md      ← Alle Tabellen & Felder
│   ├── api-endpunkte.md         ← API-Routen
│   └── rollen.md                ← Rollen & Berechtigungen
│
├── restaurant-app/              ← Hauptcodebase
│   ├── backend/                 ← Node.js + Express + TypeScript
│   │   ├── .env                 ← Umgebungsvariablen (DB, JWT, etc.)
│   │   └── src/
│   │       ├── server.ts        ← Express + Socket.io Server
│   │       ├── middleware/       ← Auth + Error Handler
│   │       ├── models/          ← Datenbank-Modelle
│   │       └── routes/          ← API-Routen
│   │
│   ├── frontend/                ← React + TypeScript + Tailwind
│   │   ├── FRONTEND.md          ← Design-Richtlinie (Farben, Abstände, Komponenten)
│   │   └── src/
│   │       ├── pages/           ← Seiten (Login, Dashboard, etc.)
│   │       ├── components/      ← UI-Komponenten
│   │       ├── hooks/           ← Custom Hooks (useTische, useBestellungen, etc.)
│   │       ├── store/           ← Zustand-Stores (Auth)
│   │       ├── lib/             ← API-Client, Supabase, Utils
```

```
│   │       └── types/          ← TypeScript-Typen
│   │
│   └── database/               ← SQL-Dateien
│       ├── schema.sql          ← Tabellen-Definitionen
│       └── seed.sql            ← Demo-Daten
│
└── prompts/                    ← Claude-Rollenbeschreibungen
    ├── assistent.md            ← Projektmanager-Rolle
    ├── arbeiter.md             ← Entwickler-Rolle
    └── sync.md                 ← Sync-Regeln
```

Nächste Schritte (Phase 4 – SaaS-Features)

- [ ] Restaurant-Registrierung & Onboarding
- [ ] Lizenzcode-System (pro Restaurant, Mitarbeiteranzahl)
- [ ] Design-Anpassung pro Restaurant
- [ ] Abonnement-Verwaltung (Mollie)